

ITALIAN VOICE ASSISTANT

V1 BUILD GUIDE

PROJECT TYPE	AI voice/text language-practice assistant
TARGET LANGUAGE	Italian
SPRINT LENGTH	3–4 weeks realistic, 2 weeks minimum build if nothing catches fire (because software enjoys ceremony).
LATENCY TARGET	Response begins in under 5 seconds during a warm session.
DEPLOYMENT TARGET	Hugging Face Spaces, Docker SDK
BUDGET	OpenAI hard cap set to €5, Hugging Face Spaces free tier.
PORTFOLIO GOAL	Show practical AI implementation, prompt design, latency awareness, testing discipline, and deployment.

1. V1 GOAL

Build a small but complete Italian conversation assistant that lets a learner practise short spoken or written exchanges.

The assistant should:

- Accept typed messages;
- Accept short voice recordings;
- Transcribe speech to text;
- Reply in Italian according to the selected level;
- Give one short correction when useful;
- Ask one short follow-up question;
- Speak the reply aloud;
- Feel responsive once the app is awake;
- Fail gracefully when something breaks.

The goal is not to build a perfect tutor. The goal is to build a reliable, limited assistant that works and can be explained clearly in a portfolio. Ancient wisdom, apparently still shocking.

2. NON-NEGOTIABLE CONSTRAINTS

Latency

Warm-session response must begin in **under 5 seconds**. Step-by-step separate measurements must be tracked:

- Recording upload complete;
- Transcription done;
- First LLM text token received;
- TTS generation started;
- First audio playback started.

Do not rely on vibes. Vibes are how bugs get tenure.

Budget

Set an OpenAI hard cap before testing heavily. Enforce structural limits:

- Explicit `max_tokens` on every LLM call;
- Short assistant replies;
- Short recording limit;
- Small conversation history;
- No unnecessary repeated TTS calls.

Scope

V1 must stay deliberately limited. Anything that does not help the first working version should be moved to V2.

3. RECOMMENDED STACK

LAYER	TOOL
Backend	FastAPI
LLM	OpenAI API, likely GPT-4o-mini or equivalent small fast model
STT	OpenAI speech-to-text
TTS	OpenAI text-to-speech
Frontend	HTML, CSS, JavaScript
Audio recording	Browser MediaRecorder API
Deployment	Hugging Face Spaces, Docker SDK
Local dev	VS Code
Version control	Git + GitHub

4. V1 ARCHITECTURE

System Components Overview:

```
Browser UI
├─ Text input
├─ Level selector
├─ Mic recording
├─ Conversation display
└─ Audio playback

FastAPI backend
├─ /chat
│   └─ sends level + history + user text to LLM
├─ /transcribe
│   └─ sends audio file to STT
└─ /speak
    └─ sends assistant reply to TTS

External services
├─ OpenAI LLM
├─ OpenAI STT
└─ OpenAI TTS
```

Basic Text Flow

```
User text → /chat → LLM streamed response → browser display
```

Basic Voice Flow

```
User speaks  
→ browser records webm blob  
→ /transcribe  
→ text sent to /chat  
→ assistant text returned  
→ /speak  
→ browser plays audio
```

For V1, full-response TTS is acceptable. Sentence-level streaming is optional only after the basic loop works.

5. SUGGESTED FOLDER STRUCTURE

```
italian-voice-assistant/  
├── api/  
│   ├── main.py  
│   ├── routes/  
│   ├── services/  
│   └── prompts/  
├── public/  
│   ├── index.html  
│   ├── app.js  
│   └── styles.css  
├── static/  
│   └── assets/  
├── scripts/  
├── logs/  
│   ├── test_log.md  
│   ├── latency_log.md  
│   └── cost_log.md  
├── .env  
├── .env.example  
├── .gitignore  
├── .dockerignore  
├── Dockerfile  
├── requirements.txt  
└── README.md
```

6. PHASE 0 — SETUP

Estimated time: Day 1

Tasks

- Create the project folder and open it in VS Code.
- Initialize Git and build a GitHub repository.
- Create boilerplate files: `README.md`, `.gitignore`, `.env.example`, `.dockerignore`, `requirements.txt`, and the `logs/` directory containing `test_log.md`, `latency_log.md`, and `cost_log.md`.
- Install core dependencies: `fastapi`, `uvicorn`, `openai`, `python-dotenv`.
- Install Docker Desktop and verify functionality.
- Set `OPENAI_API_KEY` in local `.env` file.
- Set the OpenAI hard usage cap to €5 in the OpenAI portal.

`.dockerignore` must exclude:

```
.env
__pycache__/
*.pyc
temp/
tmp/
*.webm
*.mp3
*.wav
.cache/
```

Done when:

FastAPI runs locally, Docker Desktop is running, skeleton files exist, the Git repo is configured, and the local OpenAI API key is secured and omitted from commits.

7. PHASE 1 — PERSONA AND PROMPT DESIGN

Estimated time: Days 1–2

This phase maps directly to system behavior and defines the portfolio project quality.

Assistant Behavior

Define the Italian tutor persona clearly: name, tone, target learner type, correction style, reply length, allowed languages, strictness thresholds, reactions to French/English input, and handling of broken Italian.

Core Reply Rule

One short correction if needed + one short answer/question.

The assistant must usually reply in one sentence, maximum two short sentences.

Levels Configuration

LEVEL	BEHAVIOR
Beginner	Very simple Italian, short sentences, basic foundational corrections.
Intermediate	Natural but controlled Italian, slightly richer vocabulary and expressions.
Advanced	Fully natural Italian, idioms allowed, highly precise and nuanced corrections.

Prompt Architecture

Create dedicated external files rather than embedding string literals in logic:

- `api/prompts/system_beginner.md`
- `api/prompts/system_intermediate.md`
- `api/prompts/system_advanced.md`

Playground Edge-Case Testing Checklist

Prior to coding, validate the persona in the OpenAI Playground against: beginner/intermediate errors, French/English cross-contamination, very short or excessively confused inputs, off-topic requests, and explicit translation or grammar breakdown requests.

Done when:

Persona behaves predictably across 20–30 test exchanges, prompt rules are version-controlled, and edge-case behaviors are documented.

8. PHASE 2 — BACKEND TEXT CHAT

Estimated time: Days 2–3

Endpoint

`POST /chat`

Accepts

User text, selected level, and recent conversation history.

Rules

- Validate non-empty input and enforce a strict 500-character maximum.
- Keep conversation history brief (start with 4–6 turns max).
- Set explicit `max_tokens` on every model call.
- Enable response streaming and return tokens to frontend instantly.

Done when:

Backend returns a level-appropriate, short, streamed response via curl or Postman, rejecting invalid inputs cleanly.

9. PHASE 2.5 — EARLY HOSTING SPIKE

Estimated time: Half day

Hugging Face Spaces may have cold-start delays. Warm-session performance must be verified early to ensure baseline latency targets are achievable.

- Deploy a skeleton FastAPI app to Hugging Face Spaces with a `/ping` or basic `/chat` endpoint.
- Run consecutive test requests to evaluate warm response speeds.
- Log outcomes immediately inside `logs/latency_log.md`. If warm latency fails specifications, adjust targets or infrastructure immediately before writing further logic.

10. PHASE 3 — TEXT FRONTEND

Estimated time: Days 3–4

UI Components

A minimalist layout containing: conversation display window, text input field, send button, level selector (Beginner, Intermediate, Advanced), character counter, and error notification state container.

Frontend Logic

Stream responses into UI, persist history in-memory, block inputs > 500 characters, and gracefully communicate errors (timeouts, model failures, network disconnects).

Boring foundations are still foundations, despite humanity's tragic love of fireworks. Do not proceed to audio until text is perfect.

11. PHASE 4A — VOICE BASIC LOOP

Estimated time: Days 5–7

Backend Endpoints

POST /transcribe — Accepts browser audio file (explicitly handling **webm**), rejects empty feeds, logs latency, and safely handles translation engine exceptions.

POST /speak — Accepts target text, runs TTS generation globally, outputs clean playable audio streams, and implements client-side audio caching to avoid redundant API billings on replay hits.

Frontend Voice States

Introduce mic capture triggers limited to 8–10 seconds max duration. Visually update UI through clear states: **idle**, **recording**, **transcribing**, **thinking**, **speaking**, and **error**.

Done when:

The pipeline successfully connects end-to-end (Speak → Transcribe → Chat → TTS → Playback) in under 5 seconds for a warm session.

12. PHASE 4B — OPTIONAL SENTENCE-LEVEL TTS STREAMING

Estimated time: Days 7–8 (Only if Phase 4A is completely bulletproof)

Optimize latency by breaking the LLM text output into structural sentence blocks, running concurrent TTS generations, and buffering them into a synchronized frontend queue. Keep a full fallback pipeline live. If Phase 4A already hits targets cleanly, leave this as a V2 candidate.

13. PHASE 5 — FULL FRONTEND AND UI POLISH

Estimated time: Days 8–10

- Unify text and voice features into one seamless, clean, intentional workspace.
- Incorporate deliberate, descriptive error states (e.g., *mic denied*, *empty audio*, *transcription timeout*, *model failure*).
- Avoid UI clutter, oversized animations, and redundant feature layouts. Portfolio interfaces must look clean, purposeful, and structural.

14. PHASE 6 — TESTING AND TUNING

Estimated time: Days 10–11

Run a strict verification matrix consisting of 30 test exchanges per level (90 total). Execute trap testing: silent input, rapid double-clicks, language cross-contamination (French/English), gibberish inputs, and artificially induced backend server dropouts. Document results inside the logs.

15. PHASE 7 — DOCKER AND DEPLOYMENT

Estimated time: Days 12–14

Package the app using a stable Python 3.11 base image. Ensure no temporary audio files or local `.env` configurations creep into image layers. Deploy to Hugging Face Spaces using the Docker SDK. Configure your OpenAI access key as a Space Secret, verify execution, run a definitive latency performance analysis, and complete a detailed technical project README.

16. LOGS BLUEPRINTS

logs/latency_log.md

DATE	ENVIRONMENT	STEP	TIME	NOTES
YYYY-MM-DD	local / production	recording upload complete	0.0s	
YYYY-MM-DD	local / production	transcription done	0.0s	
YYYY-MM-DD	local / production	first text token	0.0s	
YYYY-MM-DD	local / production	TTS start	0.0s	
YYYY-MM-DD	local / production	first audio playback	0.0s	

logs/test_log.md

DATE	TEST CASE	EXPECTED BEHAVIOR	ACTUAL BEHAVIOR	FIX / DECISION
YYYY-MM-DD	Beginner grammar error	Short correction + question		
YYYY-MM-DD	French input	Redirect gently to Italian		
YYYY-MM-DD	Silent recording	Useful error state shown		

logs/cost_log.md

DATE	MODEL / SERVICE	TEST VOLUME	APPROX. USAGE	NOTES
YYYY-MM-DD	LLM (GPT-4o-mini)	50 messages		
YYYY-MM-DD	STT (Whisper)	20 recordings		
YYYY-MM-DD	TTS Engine	20 replies		

17. PORTFOLIO README STRUCTURE

A comprehensive engineering README must explicitly outline: Title, Live Link, Problem Statement, Controlled V1 Scope, System Architecture Diagram, Complete Stack, Prompt Design Logic, Real Latency and Cost Metrics, Test Results, Deliberate Structural Limitations, and Local Setup Instructions.

18. DELIBERATELY OUT OF V1

Exclude to protect project timeline: Multi-session long-term memory, automated level adaptation, regional dialect packs, curated phrase databases, RAG systems, user accounts, lesson plans, analytics dashboards, and live WebRTC streaming interfaces.

19. GOOD V2 CANDIDATES

High-value future improvements: Sentence-level TTS streaming, WebRTC Realtime API integration, automated level scaling based on error frequency, persistent student memory, curated phrase packs, pronunciation feedback systems, and custom RAG notebooks.

20. FINAL ACCEPTANCE CRITERIA

V1 is complete only when: Text and voice functions perform natively; STT/TTS modules are live; level filters adjust response syntax accurately; responses enforce short correction/question loops; UI safely presents all core error states; OpenAI hard budget ceilings are active; and execution speeds track warm responses in < 5 seconds with complete log verification.

21. CORE PRINCIPLE

A working, limited assistant that fails gracefully is infinitely better than an ambitious one that never launches.

V1 should conclusively prove you can effectively scope a product, engineer a clean AI pipeline, handle full-stack integration, track latency, manage operational expenses, test edge cases, and confidently explain your architectural tradeoffs. Build real software, launch it, and prevent your project from growing antlers and escaping into the forest.